

Problem Session 08 Lock Mechanisms

- Explore more locks that are not included in the lecture.
- It is a group work of up to 6-8 students.
- Research the following mechanisms:
 - Peterson's Algorithm
 - Load-Linked and Store-Conditional
 - One additional mechanism not mentioned in the lecture
- Analyze correctness and performance for each locks using metrics discussed in class
- Create slides showing the information for each lock.
- submit your **presentation slides** (pdf) and **presentation video** (link) on LEB2.

Group: OS InW

ID of student
Name of student
Peterson's algorithm

ID of student
Name of student
X's algorithm

ID of student
Name of student
Dekker's algorithm

ID of student
Name of student
Y's algorithm

Power Point for the Lecture on
4-5 Algorithms of Software based Locks Mechanism
With 3-6 Pages per Algorithm

Template example

Test and Set Algorithm

ID of student

Name of student

Software Test and Set

- A purely software attempt at implementing a lock

```
typedef struct  lock_t { int flag; } lock_t;

void init(lock_t *mutex) {
    // 0 -> lock is available, 1 -> held
    mutex->flag = 0;
}

void lock(lock_t *mutex) {
    while (mutex->flag == 1); // TEST flag and spin-wait
    mutex->flag = 1; // now SET it!
}

void unlock(lock_t *mutex) {
    mutex->flag = 0;
}
```

What about Correctness and Performance ???

What about Correctness

Unlikely but possible



```
Thread 0
call lock()
while (flag == 1) // flag=0->continue
[INTERRUPT, SWITCH TO THREAD 1]

flag = 1; // set flag to 1 (too!)

Thread 1
call lock()
while (flag == 1) // flag=0 ...
flag = 1; // set flag to 1
[INTERRUPT, SWITCH TO THREAD 0]
```

- Both threads can enter the critical section at the same time – no mutual exclusion

What about Performance

- Spin-waiting wastes time waiting for another thread to release the lock
- On a single processor, the thread that holds the lock cannot even run (at least, not until a context switch occurs)!
- Thus, as we move forward and develop more sophisticated solutions, we should also consider ways to avoid this kind of waste